

4

Design with Material Design 3

Material Design is a design system developed by Google. It helps us create beautiful UIs. It provides a set of guidelines and components for us to use as we're developing our Android apps.

In this chapter, we'll introduce ourselves to **Material 3**. We will also cover the features that Material 3 offers. Lastly, we will learn how to use Material 3 in our Android apps and some of the components in Material 3.

In this chapter, we're going to cover the following main topics:

- Material Design 3 and its features
- Using Material Design 3 in our apps
- Building for large screens and foldables
- Making our app accessible

Technical requirements

To follow the instructions in this chapter, you will need to have Android Studio Hedgehog or later (<https://developer.android.com/studio>) downloaded.

You can find the code for this chapter at <https://github.com/PacktPublishing/Mastering-Kotlin-for-Android/tree/main/chapterfour>.

Material Design 3 and its features

The release of **Material Design 3 (Material 3)** came with lots of new features to help us build UIs for our apps. Here are some of the features of Material Design 3:

- **Dynamic color:** This is a color system that sets the color of our apps to the color of the user's wallpaper. The System UI also adapts to this color. This enables users to have that personalized feel for their apps. Please note that dynamic color only works for Android 12 and above devices.

- **More components:** Material 3 has a new set of improved components that are available for use. Some components have new UIs and others have been added to the APIs.
- **Simplified typography:** Material 3 has a much more simplified naming and grouping for typography. We have the following types: **display**, **headline**, **title**, **body**, and **label**, with each supporting *small*, *medium*, and *large* sizes. This makes it easier for us to define styles all across our apps.
- **Improved color scheme:** The color scheme has undergone a lot of improvements with the addition of more color schemes to fine-grain color customization. It's also way easier for us to support both dark and light color schemes in our apps. In addition to that, they created a new tool, Material Theme Builder (<https://m3.material.io/theme-builder>), which allows us to generate and export dark and light theme colors for our apps.
- **Simplified shapes:** Similar to typography, shapes have also been simplified to the following: **Extra Small**, **Small**, **Medium**, **Large**, and **Extra Large**. All these shapes come with default values, which we can always override to use our own.

The good news for us is that from Android Studio Hedgehog onward, we have project templates that come set up with Material 3, which makes things easier for us. Even the project we created in *Chapter 2* comes with Material 3 set up already.

Material 3 APIs and their predecessors offer a wide range of components for us to use in our apps. In the next subsection, we will be looking at some of the common ones.

Material components

The Material library comes with prebuilt components that we can use to build common UI components. Let us look at some of the commonly used components and some of the updates they had in Material 3.

Top app bars

This is a component displayed at the top of the screen. It has a title and can also have some actions that are related to the screen the user is on. Some of the common actions are the settings icon normally at the top right of the screen. In Material 3, we have four types of top app bars: **center-aligned**, **small**, **medium**, and **large**, as shown in the following figures.



Figure 4.1 – Small top app bar



Figure 4.2 – Center-aligned top app bar



Figure 4.3 – Medium top app bar



Figure 4.4 – Large top app bar

As seen in *Figures 4.1 to 4.4*, all the top bars have the same width and only differ in height and positioning of the title text.

Let us look at the sample code for one of these top app bars:

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun PacktCenterAlignedTopBar() {
    CenterAlignedTopAppBar(
        title = {
            Text(text = "Packt Publishing")
        }
    )
}
```

Here, we have our custom composable and, inside it, we are using the `CenterAlignedTopBar` composable from Material 3 and passing in `Text` to the `title` composable. The other three (`LargeTopAppBar`, `MediumTopAppBar`, and `TopAppBar`) are similar; the only difference is the composable that you will use. Notice that we have the `@OptIn` annotation as these components are still experimental.

Next, let us look at the `FloatingActionButton` component.

FloatingActionButton

Most apps use the component to represent a call to action that is frequently used in the app. For example, create a new chat in a chat app. It is normally positioned at the bottom right of the screen or elsewhere, depending on your use case. This is how we create the component:

```
FloatingActionButton(  
  onClick = { /*TODO*/ },  
  content = {  
    Icon(  
      imageVector = Icons.Default.Add ,  
      contentDescription = "New Chat"  
    )  
  }  
)
```

We use the `FloatingActionButton` component from the Material 3 library. We have the `onClick` argument on the composable and, inside the `content` lambda, we pass in an `Icon` composable that has an `add` icon. The preview should be the following:



Figure 4.5 – FloatingActionButton

The `FloatingActionButton` component has these sizes: large, normal, and small, and you can use whichever fits your purpose.

We have another type of `FloatingActionButton` component known as `ExtendedFloatingActionButton`, which looks like this:

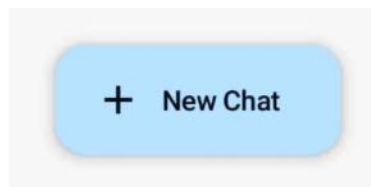


Figure 4.6 – ExtendedFloatingActionButton

As seen in the preceding figure, an `ExtendedFloatingActionButton` component allows us to add more items to our FAB. They are wider than the normal `FloatActionButton` components. In this case, we have a `Text` composable with the text **New Chat** in addition to the icon. You can use it with or without the icon. The implementation for this is as follows:

```
ExtendedFloatingActionButton(
    onClick = { /*TODO*/ },
    content = {
        Icon(
            imageVector = Icons.Default.Add ,
            contentDescription = "New Chat"
        )
        Text(
            modifier = Modifier.padding(10.dp) ,
            text = "New Chat"
        )
    }
)
```

Here, we used the `ExtendedFloatingActionButton` component and still passed in the same parameters as before. The only difference is that inside the content, we pass in a text since the content lambda exposes `RowScope`, which means children composables will be arranged in a row.

Next, let us look at the bottom app bar components.

Bottom app bars

The bottom app bar components display navigation items at the bottom of the screen. They are normally useful for apps that have three to five primary destinations.

Let us look at the code for a bottom app bar:

```
BottomAppBar(
    actions = {
        Icon(imageVector = Icons.Rounded.Home, contentDescription =
"Home Screen")
        Icon(imageVector = Icons.Rounded.ShoppingCart,
contentDescription = "Cart Screen")
        Icon(imageVector = Icons.Rounded.AccountCircle,
contentDescription = "Account Screen")
    }
)
```

We use the `BottomAppBar` component and, inside the `actions` lambda, we pass in three `Icon` composables to represent the items we are supposed to show. This is how the preview of the composable will look:



Figure 4.7 – BottomAppBar

In *Figure 4.7*, we can see we have three icons arranged horizontally.

Additionally, in `BottomAppBar`, we can also provision a `FloatingActionButton` component. We are going to use the `FloatingActionButton` component that we used early on. The updated component code is as follows:

```
BottomAppBar(
  actions = {
    Icon(imageVector = Icons.Rounded.Home, contentDescription =
      "Home Screen")
    Icon(imageVector = Icons.Rounded.ShoppingCart,
      contentDescription = "Cart Screen")
    Icon(imageVector = Icons.Rounded.AccountCircle,
      contentDescription = "Account Screen")
  },
  floatingActionButton = {
    PacktFloatingActionButton()
  }
)
```

In the preceding code, we have used the `floatingActionButton` parameter and passed in `PacktFloatingActionButton()` that we created earlier. The updated preview will be as follows:



Figure 4.8 – BottomAppBar with FloatingActionButton

As seen in the preceding figure, our `BottomAppBar` now has a nice `FloatingActionButton` to its right. The FAB is automatically positioned for you to the right.

We have looked at the different components in isolation, but what happens when we want to place them on one screen together? Next, we are going to look at `Scaffold`, which is meant for this.

Scaffold

This is a layout provided by Material Design that helps place all components on your screen in their desired positions with ease.

Let us look at a sample of `Scaffold` that has a top app bar, a floating action button, text center-aligned on the screen, and a bottom navigation bar:

```
Scaffold(
    topBar = {
        PacktSmallTopAppBar()
    },
    bottomBar = {
        PacktBottomNavigationBar()
    },
    floatingActionButton = {
        PacktFloatingActionButton()
    },
    content = { paddingValues ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(paddingValues)
                .background(Color.Gray.copy(alpha = 0.1f)),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text(
                modifier = Modifier.padding(10.dp),
                text = "Mastering Kotlin for Android Development -
Chapter 4",
                textAlign = TextAlign.Center
            )
        }
    }
)
```

A lot is happening here, so let us break it down:

- The `Scaffold` composable is used to create a layout that implements the Material Design guidelines. It is a container that contains the top bar, bottom bar, floating action button, and the content.
- The `topBar` parameter is used to specify the top bar. In this case, we are using the `PacktSmallTopAppBar` composable that we created earlier.
- The `bottomBar` parameter is used to specify the bottom bar. In this case, we are using the `PacktBottomNavigationBar` composable.
- The `floatingActionButton` parameter is used to specify the floating action button. In this case, we are using the `PacktFloatingActionButton` composable.

- The `content` parameter is used to specify the content of the screen. In this case, we are using a `Column` composable that contains a `Text` composable. The text is centered in the column using the `verticalArrangement` and `horizontalAlignment` parameters. Notice that inside `Column`, we are using the `paddingValues` parameter to add padding to the column. This is because the `Scaffold` composable passes the padding values to the `content` parameter.

With our `Scaffold` composable ready, let us see how its preview looks:



Figure 4.9 – Scaffold

In *Figure 4.9*, we can see that the `Scaffold` composable has added the top bar, bottom bar, and the floating action button to the screen. The components are also placed in the correct positions as per the Material Design guidelines.

We have learned about a bunch of components so far; Material 3 still offers more components out of the box for us. We will be using some of these components in the upcoming chapters of this book. To view the full list of all the components, go to the Material 3 Components website (<https://m3.material.io/components>) to see them and their guidelines.

Now that we understand Material 3 and its features, let us look at how to add it to our apps.

Using Material Design in our apps

To take advantage of the Material 3 features that we looked at in the previous section, we need to add it to our app. Luckily, with Android Studio Hedgehog, we have Material 3 templates. Even the sample apps we have been using already use Material 3. So cool, right? Let us quickly scan the dependencies to understand what is happening:

```
implementation 'androidx.core:core-ktx:1.10.1'
implementation platform('org.jetbrains.kotlin:kotlin-bom:1.8.0')
implementation 'androidx.lifecycle:lifecycle-runtime-ktx:2.6.1'
implementation 'androidx.activity:activity-compose:1.7.2'
implementation platform('androidx.compose:compose-bom:2022.10.00')
implementation 'androidx.compose.ui:ui'
implementation 'androidx.compose.ui:ui-graphics'
implementation 'androidx.compose.ui:ui-tooling-preview'
implementation 'androidx.compose.material3:material3'
testImplementation 'junit:junit:4.13.2'
androidTestImplementation 'androidx.test.ext:junit:1.1.5'
androidTestImplementation 'androidx.test.espresso:espresso-core:3.5.1'
androidTestImplementation platform('androidx.compose:compose-bom:2022.10.00')
androidTestImplementation 'androidx.compose.ui:ui-test-junit4'
debugImplementation 'androidx.compose.ui:ui-tooling'
debugImplementation 'androidx.compose.ui:ui-test-manifest'
```

This dependencies block sets up several for us, including the compose dependencies. The most important one is the `androidx.compose.material3:material3` dependency. This is the dependency that contains the Material 3 components. We are using the Compose **Bill of Materials (BOM)** to manage our dependencies. This means that we do not have to specify the version of each dependency. Instead, we specify the version of the BOM, and it will manage the versions of the dependencies for us. This is the recommended way of managing dependencies in Compose. That is why we have not specified the version of each dependency.

With that, our project is ready to take advantage of the Material 3 features. In the next subsection, we will be adding more color schemes to the app.

Adding Material Design 3 color schemes

As mentioned earlier, Material 3 comes with a lot of fine-grained color schemes and introduces **dynamic color**. However, they are not set up with the templates that Android Studio generates. We will be setting them up in the next few steps.

Head over to the `ui/theme` package and open the `Color.kt` file, which has the following code:

```
val Purple80 = Color(0xFFD0BCFF)
val PurpleGrey80 = Color(0xFFCCC2DC)
val Pink80 = Color(0xFFE8BFD0)

val Purple40 = Color(0xFF6650a4)
val PurpleGrey40 = Color(0xFF625b71)
val Pink40 = Color(0xFF7D5260)
```

So far, this file only has a few colors defined. These colors do not cover all the color tokens provided by Material 3. We will add more colors as we need them in the app.

We will be using the **Material Theme Builder** tool to generate these colors. Let us open our browser and go to the Material Theme Builder tool (<https://m3.material.io/theme-builder>). We will be presented with the following screen:

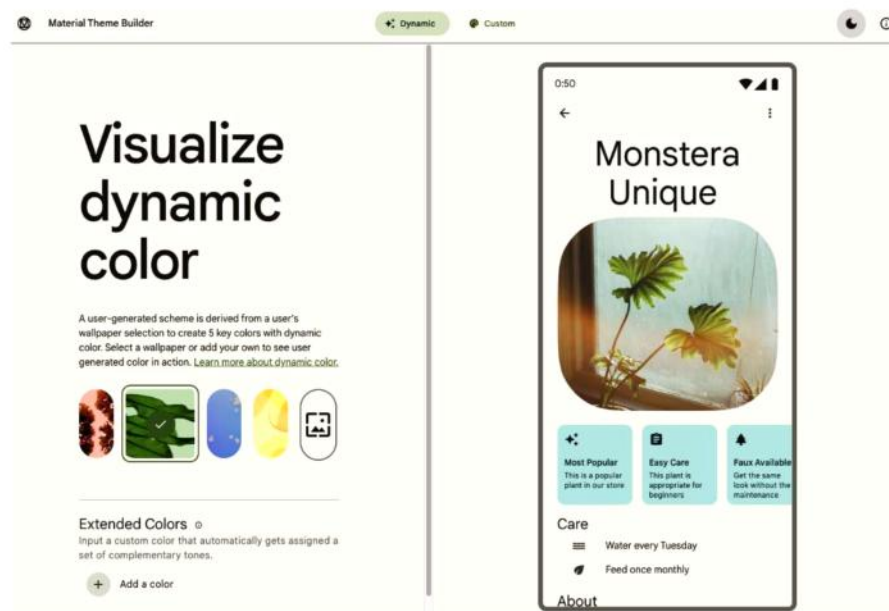


Figure 4.10 – Material Theme Builder tool

This tool helps us visualize the color schemes for our apps and shows how different components will be themed. It makes it easier for us to customize and produce a consistent color scheme for our app. It has two tabs: **Dynamic** and **Custom**. In the **Dynamic** tab, we can select one of the preloaded colors or wallpapers to see how the color changes. One useful feature is that we can also add your own wallpaper and generate the colors based on the wallpaper.

In the **Custom** tab, we can select a color and the tool generates all the complementary colors for us based on the color we select, ensuring a harmonious color palette:

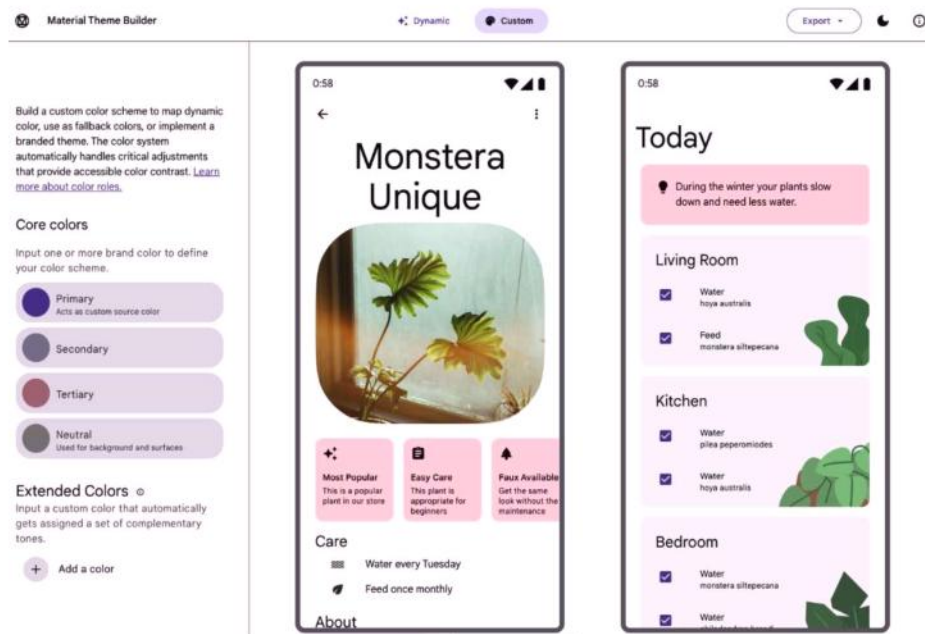


Figure 4.11 – Material Theme Builder tool custom color

On the left, we have the **Core colors** section where we can select either **Primary**, **Secondary**, **Tertiary**, or **Neutral** colors for our app.

We are going to select the **Primary** color option, which opens a **Color Picker** dialog:

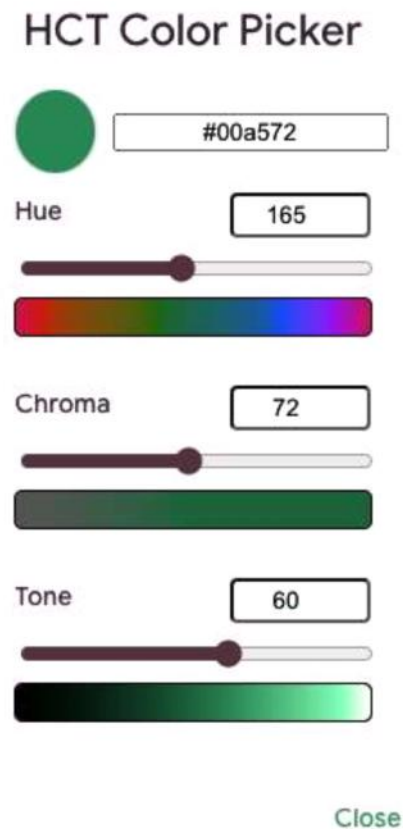


Figure 4.12 – Color Picker

As we are changing the primary color, we will be able to see the visual preview change to match the color we have. Now we have our primary color ready, we are going to export the files so that we can use them in our project:

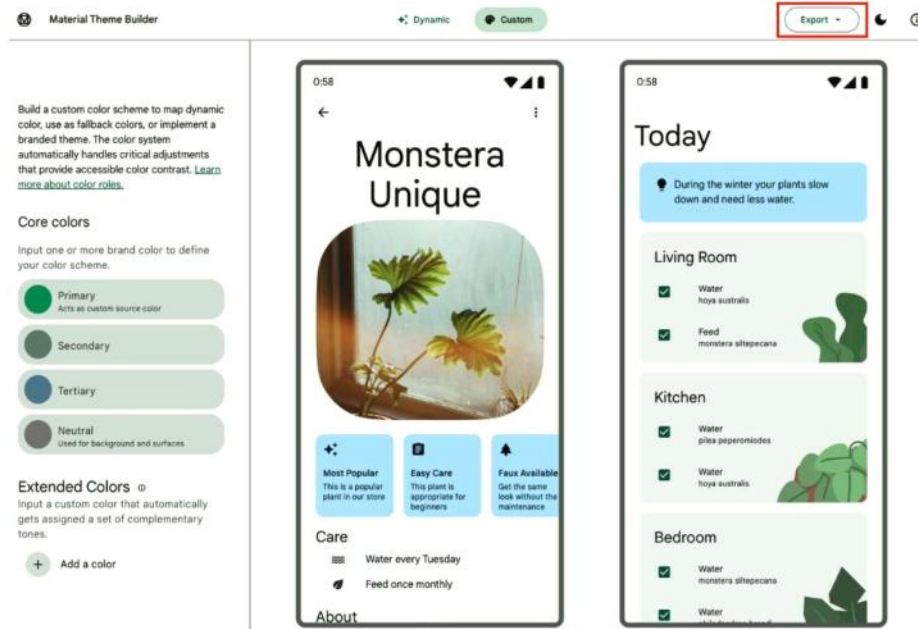


Figure 4.13 – Export option

As seen in *Figure 4.13*, the visual preview has changed to the color we chose. Click on the **Export** option (highlighted in *Figure 4.13*) and, from the dropdown that appears, choose **Jetpack Compose (Theme.kt)**. This will prompt a download of a zipped file. Unzip the file and open the folder. We will now have a `ui` folder that has a `theme` folder inside containing `Color.kt` and `Theme.kt` files.

Let us open the `Color.kt` file, which contains the following code:

```
val md_theme_light_primary = Color(0xFF006C49)
val md_theme_light_onPrimary = Color(0xFFFFFFFF)
val md_theme_light_primaryContainer = Color(0xFF7AFAC0)
val md_theme_light_onPrimaryContainer = Color(0xFF002113)
val md_theme_light_secondary = Color(0xFF4D6357)
val md_theme_light_onSecondary = Color(0xFFFFFFFF)
val md_theme_light_secondaryContainer = Color(0xFFD0E8D8)
val md_theme_light_onSecondaryContainer = Color(0xFF0A1F16)
val md_theme_light_tertiary = Color(0xFF3D6473)
val md_theme_light_onTertiary = Color(0xFFFFFFFF)
val md_theme_light_tertiaryContainer = Color(0xFFC0E9FB)
val md_theme_light_onTertiaryContainer = Color(0xFF001F29)
```



```
val md_theme_light_error = Color(0xFFBA1A1A)
val md_theme_light_errorContainer = Color(0xFFFFDAD6)
val md_theme_light_onError = Color(0xFFFFFFFF)
val md_theme_light_onErrorContainer = Color(0xFF410002)
val md_theme_light_background = Color(0xFFFFBDF9)
val md_theme_light_onBackground = Color(0xFF191C1A)
val md_theme_light_surface = Color(0xFFFFBDF9)
val md_theme_light_onSurface = Color(0xFF191C1A)
val md_theme_light_surfaceVariant = Color(0xFFDCE5DD)
val md_theme_light_onSurfaceVariant = Color(0xFF404943)
val md_theme_light_outline = Color(0xFF707973)
val md_theme_light_inverseOnSurface = Color(0xFFE8E8E8)
val md_theme_light_inverseSurface = Color(0xFF2E312F)
val md_theme_light_inversePrimary = Color(0xFF5CDDA5)
val md_theme_light_shadow = Color(0xFF000000)
val md_theme_light_surfaceTint = Color(0xFF006C49)
val md_theme_light_outlineVariant = Color(0xFFC0C9C1)
val md_theme_light_scrim = Color(0xFF000000)

val md_theme_dark_primary = Color(0xFF5CDDA5)
val md_theme_dark_onPrimary = Color(0xFF003824)
val md_theme_dark_primaryContainer = Color(0xFF005236)
val md_theme_dark_onPrimaryContainer = Color(0xFF7AFAC0)
val md_theme_dark_secondary = Color(0xFFB4CCBD)
val md_theme_dark_onSecondary = Color(0xFF20352A)
val md_theme_dark_secondaryContainer = Color(0xFF364B40)
val md_theme_dark_onSecondaryContainer = Color(0xFFD0E8D8)
val md_theme_dark_tertiary = Color(0xFFA5CDDE)
val md_theme_dark_onTertiary = Color(0xFF063543)
val md_theme_dark_tertiaryContainer = Color(0xFF244C5A)
val md_theme_dark_onTertiaryContainer = Color(0xFFC0E9FB)
val md_theme_dark_error = Color(0xFFFFB4AB)
val md_theme_dark_errorContainer = Color(0xFF93000A)
val md_theme_dark_onError = Color(0xFF690005)
val md_theme_dark_onErrorContainer = Color(0xFFFFDAD6)
val md_theme_dark_background = Color(0xFF191C1A)
val md_theme_dark_onBackground = Color(0xFFE1E3DF)
val md_theme_dark_surface = Color(0xFF191C1A)
val md_theme_dark_onSurface = Color(0xFFE1E3DF)
val md_theme_dark_surfaceVariant = Color(0xFF404943)
val md_theme_dark_onSurfaceVariant = Color(0xFFC0C9C1)
val md_theme_dark_outline = Color(0xFF8A938C)
val md_theme_dark_inverseOnSurface = Color(0xFF191C1A)
```

```

val md_theme_dark_inverseSurface = Color(0xFFE1E3DF)
val md_theme_dark_inversePrimary = Color(0xFF006C49)
val md_theme_dark_shadow = Color(0xFF000000)
val md_theme_dark_surfaceTint = Color(0xFF5CDDA5)
val md_theme_dark_outlineVariant = Color(0xFF404943)
val md_theme_dark_scrim = Color(0xFF000000)

```

As you can now see, we have more color additions. Let us copy these colors to the `Color.kt` file in our project. Next, let us open the `Theme.kt` from the unzipped folder.

We will notice it is similar to the `Theme.kt` file but it defines all the Material 3 color schemes. Copy the contents of this file and paste them into the `Theme.kt` file in our project. We are going to make minor edits to the code to make sure we maintain the `ChapterFourTheme` name and dynamic color logic that was in our theme. Next, we need to change the `DarkColorScheme` variable values to the following:

```

private val DarkColorScheme = darkColorScheme(
    primary = md_theme_light_primary,
    onPrimary = md_theme_light_onPrimary,
    primaryContainer = md_theme_light_primaryContainer,
    onPrimaryContainer = md_theme_light_onPrimaryContainer,
    secondary = md_theme_light_secondary,
    onSecondary = md_theme_light_onSecondary,
    secondaryContainer = md_theme_light_secondaryContainer,
    onSecondaryContainer = md_theme_light_onSecondaryContainer,
    tertiary = md_theme_light_tertiary,
    onTertiary = md_theme_light_onTertiary,
    tertiaryContainer = md_theme_light_tertiaryContainer,
    onTertiaryContainer = md_theme_light_onTertiaryContainer,
    error = md_theme_light_error,
    errorContainer = md_theme_light_errorContainer,
    onError = md_theme_light_onError,
    onErrorContainer = md_theme_light_onErrorContainer,
    background = md_theme_light_background,
    onBackground = md_theme_light_onBackground,
    surface = md_theme_light_surface,
    onSurface = md_theme_light_onSurface,
    surfaceVariant = md_theme_light_surfaceVariant,
    onSurfaceVariant = md_theme_light_onSurfaceVariant,
    outline = md_theme_light_outline,
    inverseOnSurface = md_theme_light_inverseOnSurface,
    inverseSurface = md_theme_light_inverseSurface,
    inversePrimary = md_theme_light_inversePrimary,

```



```

        surfaceTint = md_theme_light_surfaceTint,
        outlineVariant = md_theme_light_outlineVariant,
        scrim = md_theme_light_scrim,
    )

```

In the preceding code, we are using the `DarkColorScheme` function to create a dark color scheme. We are passing the colors that we generated using the Material Theme Builder tool. We will use this color scheme to create a dark theme. The dark color scheme variable is defined in a similar manner, and we can copy the values from the `Theme.kt` file from the tool and add it there too. Let us now look at our theme composable in whole:

```

@Composable
fun ChapterFourTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S
    } -> {
        val context = LocalContext.current
        if (darkTheme) dynamicDarkColorScheme(context) else
        dynamicLightColorScheme(context)
    }

    darkTheme -> DarkColorScheme
    else -> LightColorScheme
    }
    val view = LocalView.current
    if (!view.isInEditMode) {
        SideEffect {
            val window = (view.context as Activity).window
            window.statusBarColor = colorScheme.primary.toArgb()
            WindowCompat.getInsetsController(window, view).
            isAppearanceLightStatusBars = darkTheme
        }
    }

    MaterialTheme(
        colorScheme = colorScheme,
        typography = Typography,
        content = content
    )
}

```

Let us break down the preceding code:

- The `ChapterFourTheme` composable is used to create a theme for our app. It takes three parameters:
 - The `darkTheme` parameter is used to specify whether the theme is dark or light. By default, we are using the system theme.
 - The `dynamicColor` parameter is used to specify whether the theme should use dynamic colors. By default, we are using dynamic colors.
 - The `content` parameter is used to specify the content of the theme. In this case, we are using the `MaterialTheme` composable to create a theme for our app.
- The `colorScheme` variable is used to specify the color scheme to use. We are using a `when` expression to determine the color scheme to use. If the `dynamicColor` parameter is `true` and the device is running Android 12 or higher, we are using the `dynamicDarkColorScheme` or `dynamicLightColorScheme` function to create a dynamic color scheme. When not using dynamic colors, we fall back to the normal themes. If the `darkTheme` parameter is `true`, we are using the `DarkColorScheme` variable to create a dark color scheme. Otherwise, we are using the `LightColorScheme` variable to create a light color scheme.
- The `view` variable is used to get the view that is using the theme.
- The `SideEffect` composable is used to execute a side effect. In this case, we are using it to set the status bar color and the status bar icons' color. We are using the `WindowCompat` class to get `InsetsController` and set the status bar color and the status bar icons' color. We are using the `colorScheme.primary` color to set the status bar color. We are using the `darkTheme` parameter to determine whether the status bar icons' color should be light or dark.
- The `MaterialTheme` composable is used to create a theme for our app. We are using the `colorScheme` parameter to specify the color scheme to use. We are using the `typography` variable to specify the typography to use.

To be able to see the changes we have made, we need to call `PacktScaffold`, which we created earlier inside the `MainActivity.kt` file in the `ChapterFourTheme` block:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            ChapterFourTheme {
                PacktScaffold()
            }
        }
    }
}
```

Let us build and run the app. We should be able to see the following:

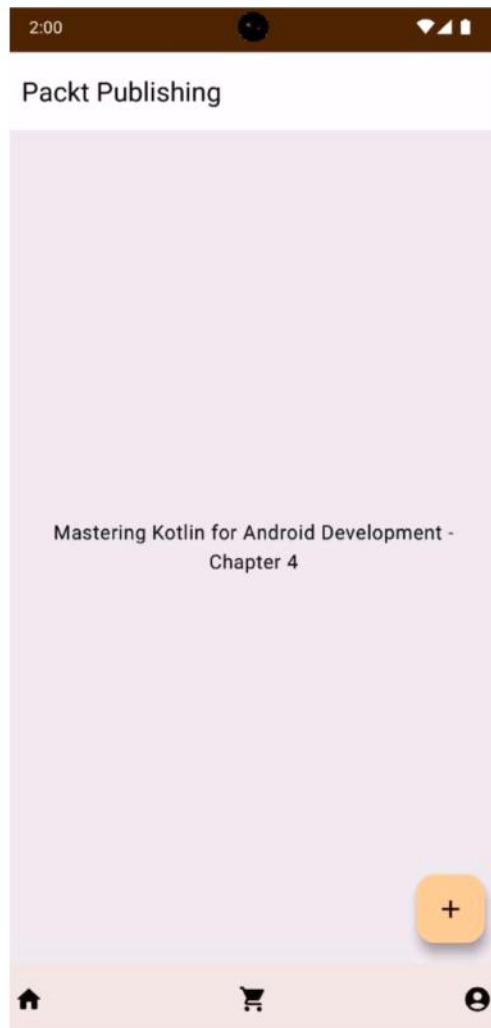


Figure 4.14 – Chapter 4 app

Like me, you might be wondering why the app does not have the greenish color we set on the Material Theme Builder tool. Remember the dynamic color logic in our `ChapterFourTheme` composable? It is responsible for the brownish color we see. Look at my wallpaper settings:



Figure 4.15 – Wallpaper settings

As seen in *Figure 4.15*, my wallpaper has a brown color set. This means our dynamic color logic is working, and our app can adapt well to my wallpaper settings!

We have seen how to use the Material 3 features in our app. In the next section, we will see how we can design UIs that work for large screens and foldables in Jetpack Compose.

Designing UIs for large screens and foldables

In recent years and with the release of Material 3, there has been an increased focus on tablets and foldable devices. As such, we, as developers, must make our apps work well on these devices. In this section, we will look at how we can make our apps work well on large screens and foldables. We need to ensure our apps adapt to the different screen sizes. Making our apps responsive provides a good user experience.

Material 3 offers **canonical layouts** to serve as guidelines for creating UIs for large screens and foldables. These layouts are as follows:

- **List-detail view:** Here, we place a list of items on the left and, on the right, we show the details of a single item.
- **Feed:** Here, we arrange content elements such as cards in a customizable grid, which provides a good view of a large amount of content.
- **Supporting pane:** Here, we organize app content into primary and secondary display areas. The primary area shows the main content while the secondary area shows the supporting content. The primary area occupies most of the screen while the secondary area occupies a smaller portion.

For us to show the different layouts, we must know the screen size of the device the user is using. Luckily for us, Jetpack Compose provides us with a way to get the screen size. We have the Material 3 `WindowSizeClass` to help us determine which layout to show in our app.

We are going to learn how to use `WindowSizeClass` next.

Using WindowSizeClass

For us to use `WindowSizeClass`, we must add the following dependency to our app:

```
implementation 'androidx.compose.material3:material3-window-size-class'
```

This is the Material 3 dependency that adds `WindowSizeClass` to our project. Notice we are not providing a version for this dependency. This is because we are using the Compose BOM to manage our dependencies. The BOM will manage the version of this dependency for us.

`WindowSizeClass` classifies the available screen width into three categories:

- **Compact:** This is for devices whose width is less than 600 dp. Commonly, these are devices in portrait mode.
- **Medium:** This is for devices whose width is between 600 dp and 840 dp. Devices such as tablets and foldables in portrait mode fall into this category.
- **Expanded:** This is for devices whose width is greater than 840 dp. Devices such as tablets and foldables in landscape mode, phones in landscape mode, and desktops fall into this category.

WindowSizeClass uses widthSizeClass to get the width of the screen. In addition to widthSizeClass, it also has heightSizeClass to help us determine the height of the screen.

Let us look at widthSizeClass in action:

```
when(calculateWindowSizeClass(activity = this).widthSizeClass) {
    WindowWidthSizeClass.Compact -> {
        CharactersScreen(
            navigationOptions = NavigationOptions.BottomNavigation,
            showDetails = false
        )
    }
    WindowWidthSizeClass.Medium -> {
        CharactersScreen(
            navigationOptions = NavigationOptions.NavigationRail,
            showDetails = true
        )
    }
    WindowWidthSizeClass.Expanded -> {
        CharactersScreen(
            navigationOptions = NavigationOptions.NavigationDrawer,
            showDetails = true
        )
    }
    else -> {
        CharactersScreen(
            navigationOptions = NavigationOptions.BottomNavigation,
            showDetails = false
        )
    }
}
```

Here is an explanation of what the preceding code is doing:

- The calculateWindowSizeClass function is used to calculate WindowSizeClass. We are passing the activity as the parameter. The function is from the WindowSizeClass APIs. It has the widthSizeClass and heightSizeClass properties, which we can use to get the width and height of the screen, respectively.
- We use widthSizeClass to customize our display options:
 - For the WindowWidthSizeClass.Compact case, we are using BottomNavigation for navigation, and the UI should only show the list of characters
 - For the WindowWidthSizeClass.Medium case, we are using NavigationRail for navigation, and the UI should show the list of characters and the details of the selected character

- For the `WindowWidthSizeClass`. Expanded case, we are using `NavigationDrawer` for navigation. and the UI should show the list of characters and the details of the selected character
- We have a default case where we are using `BottomNavigation` for navigation, and the UI should only show the list of characters

At a glance, you can see how we take advantage of `WindowSizeClass` to customize our UI and navigation type based on the screen size. This is an immensely powerful feature that we can use to make our apps responsive. This makes sure we take advantage of the screen size and provide a good user experience.

The example shown in this section was a simple one. In *Chapter 7* of this book, we will have a more detailed example where we will use `WindowSizeClass` to customize our UI based on the screen size.

Now that we know how to design and build apps for large screens and foldables, let us look at yet another important topic in this chapter, which is accessibility.

Making our app accessible

Making the apps that we develop accessible is extremely important. It ensures that our apps can be used by everyone. In this section, we will look at how we can make our apps accessible. Jetpack Compose uses **semantics** to make our apps accessible. Semantics are used to describe the UI elements in our apps. They are used by accessibility services to make our apps accessible. Semantics are also used by automated testing tools to test our apps. Some of the best practices for making our apps accessible are as follows:

- We should always ensure that all clickable or touchable elements or those that require user interaction are large enough to be easily tapped or clicked. Most Material components out of the box have a default size that is large enough to be easily tapped or clicked. If we must size by ourselves, we should ensure that the size is at least 48 dp by 48 dp.
- We should add content descriptions to our composables. Components such as `Icon` and `Image` provide this argument to describe visual elements to accessibility services. We should always provide a content description for these components. The following is an example:

```
Icon(  
    modifier = Modifier.size(48.dp),  
    painter = painterResource(id = R.drawable.ic_launcher_  
        foreground),  
    contentDescription = "Icon"  
)
```

You can see in the preceding code that we are using the `contentDescription` parameter to provide a description for `Icon`. This is a good practice that we should always follow.

- We should label our clickable elements. We can pass a clickable label to the clickable modifiers. This enables us to add descriptions to our clickable elements. The following is an example:

```
Text (
    modifier = Modifier
        .clickable(
            onClick = { /*TODO*/ },
            onClickLabel = "Click Me"
        )
        .padding(10.dp),
    text = "Click Me"
)
```

In the preceding example, we are using the `onClickLabel` parameter to add a description to the `Text` composable. This is a good practice that we should always follow.

- By using semantics, we can also describe headers. Headers are used to describe the content that follows them. We can use semantics to add a header to our composables. The following is an example:

```
Text (
    modifier = Modifier
        .semantics { heading() }
        .padding(10.dp),
    text = "Heading One"
)
```

- We can additionally provide information about the state of our composables. For example, we can provide information about the state of a button. We can use semantics to provide this information. The following is an example:

```
Button(
    modifier = Modifier
        .semantics { stateDescription = "Disabled" }
        .padding(10.dp),
    onClick = { /*TODO*/ },
    enabled = false
) {
    Text(text = "Disabled Button")
}
```

This helps us inform our users about the state of our composables.

- For some groups of components, we can also use the `mergeDescendants` parameter to merge the semantics of the children composables. The following is an example:

```
Column(  
    modifier = Modifier  
        .padding(10.dp)  
        .semantics(mergeDescendants = true) { }  
) {  
    Text(text = "Heading One")  
    Text(text = "Heading Two")  
    Text(text = "Heading Three")  
}
```

Merging descendants is useful when we want to provide a description for a group of composables. In the preceding example, we are using the `mergeDescendants` parameter to merge the semantics of the `Text` composables. However, we should be careful when using this parameter. We should only use it when we want to provide a description for a group of composables. If we use it for a large group of composables, it can lead to performance issues and can also be confusing for the users.

To learn more about accessibility in Jetpack Compose, visit the official documentation (<https://developer.android.com/jetpack/compose/accessibility>).

Summary

In this chapter, we introduced Material 3. We also covered the features that Material 3 offers. We saw how to use Material 3 in our apps. Additionally, we covered how to design and develop our apps for large screens and, finally, we saw how to make our Jetpack Compose UIs accessible.

In the next chapter, we will continue building upon the skills that we've gained from the previous chapters. We will be looking at how to architect our app and the different architectures available. We will learn how to use Jetpack libraries in our apps and how to tackle dependency injection.